



A Framework for Integrating Architectural Design Patterns into PCG

Arunpreet Sandhu
Computer Science
University of California, Davis
Davis, CA, USA
asisandhu@ucdavis.edu

Joshua McCoy
Computer Science & Cinema and Digital Media
University of California, Davis
Davis, CA, USA
jamccoy@ucdavis.edu

ABSTRACT

Asset generation has a huge time cost associated with it in games. In order to reduce this time cost, designers have adopted Procedural Content Generation (PCG) systems into their workflows. However, most PCG techniques may only give a modest workflow speed up. While some PCG techniques can give immense speedups, they have extra time costs that might not be worth it. These techniques require users to encode their domain knowledge in a way these techniques can understand. This can be a costly process and the technique might not be able to be reused in another project. We propose a framework for encoding domain knowledge in a way to promote reusability. We provide an example that is based on architectural design philosophies.

CCS CONCEPTS

• **Applied computing** → **Computer games** • **Theory of computation** → **Constraint and logic programming** • **Computing methodologies** → **Probabilistic reasoning**

KEYWORDS

Procedural Content Generation, Wave Function Collapse, Level Generation

1 Introduction

Designing assets for games can be a daunting task due to intensive design time. This large time to asset design ratio has led developers to use algorithmic approaches to help reduce design time [1]. An example of such is using Procedural Content Generation (PCG) to reduce design time by offloading work to a machine [2]. However, most of these algorithms are incapable of

autonomously generating assets due to their lack of domain knowledge [3]. An analogy would be if an automotive designer was asked to design an aircraft. The automotive designer would say they can't design an aircraft because of their lack of aircraft design. This domain-specific knowledge is what allows for a specialized design, such as assets for games. Without this deep understanding of a domain, almost all PCG algorithms will be only capable of generating generic assets. Luckily, this can be an advantage to designers.

There are different ways to apply PCG algorithms to help reduce design time. One such method is generating generic assets to be refined. This method gives the designer a generic asset to refine into a unique piece that will fit into the game world. This concept of a mixed workflow has been prevalent within the industry for years to speed up design. This workflow is by far one of the most commonly used, other methods are sparingly used due to their unpredictable nature. Other methods require some domain knowledge to be encoded into the PCG algorithm. The payoff of the encoding is the PCG algorithm can generate more nuanced assets that require less to no refinement [4]. However, these approaches beg the question of if the upfront cost was worth the time savings that are seen later within the workflow.

While having no refinement seems enticing, it can lead to either immense amount of upfront costs that may not make the savings worth it or still generate generic assets. The reason comes from the something mentioned before; machines aren't good at answering why to a question. In games, being able to answer the why leads to more believable worlds. Believability comes from interweaving different parts together within a game. In order to interweave, designers must be aware of the different parts. Suddenly, the domain knowledge that needs to be encoded seems to have grown out of control that it wouldn't make sense to encode domain knowledge into a PCG algorithm. Instead, encoding some of the domain knowledge can give designers a way to lower time costs and reuse these PCG algorithms for other projects. However, the question of what should be encoded is a tricky one, which we try to give a solution to.

In this work, we give a framework for building a foundational step in creating more reusable domain encoded PCG algorithms. We do so by first picking out what we believe to be candidate concepts for encoding from architecture. After choosing these

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the Owner/Author.

FDG '19, August 26–30, 2019, San Luis Obispo, CA, USA
© 2019 Copyright is held by the owner/author(s).
ACM ISBN 978-1-4503-7217-6/19/08.
<https://doi.org/10.1145/3337722.3341839>

candidate concepts, we will describe a way to use a commercial PCG algorithm to encode that domain knowledge.

2 Related Works

Encoding domain knowledge into an algorithm is not a new idea by any means. Works such as Cyc [5], [6] are examples of this domain encoding. Started in 1984, Cyc is one of the longest running AI projects [7] with the goal of encoding common sense. To do this, the creators of Cyc use a rules-based approach in order to encode knowledge. By using simple facts to build up a knowledge base such as, “birds have wings, a parrot is a bird,” the system can reason over this small knowledge space of a parrot. Asking the system, “do parrots have wings,” the system would be able to output something such as, “a parrot has wings.” While trivial, this example showcases how these approaches can infer about the information they have. If the information does not exist, the result would be unsatisfiable since the system has no prior knowledge.

Cyc has over one thousand years of cumulative work on encoding these facts. A terrifying number in terms of cost, considering some game developers can’t give one thousand years to rule encoding. Fortunately, most systems that are used for asset generation do not require the knowledge base that Cyc has. Systems instead focus on having enough facts to generate something that can satisfy whatever requirements the designer needs. Yet, the requirements that designer needs can change depending on the game they are working on. In which case the designer might have to encode their own knowledge into these algorithms. The problem this poses is the designer might not have the technical background to use these systems. The designer now either spends even more time in order to use these systems for a modest speed up or doing the work without any assistance. These are the options in a rule-based system, however, there are non-rule-based approaches to PCG systems.

Non-rule based PCG systems that use other methods use random choice for asset generation [8]. They only require some data about what they are generating. An example would be a map generator that takes in a tileset and outputs a tile map. The issue with these approaches is that the output might be horrendously random, like a tile map that has no reason why it was put together that way. However, there are ways to augment these approaches to make sure that all outputs make some sense given the context. An example of this would be the PCG system within *Minecraft* [9]. Interestingly enough, the gameplay of *Minecraft* parallels the PCG augmented workflow. *Minecraft* is a game about taking the terrain and creating something unique out of it. With *Minecraft* being an exception, the issue with this approach is that the system’s output must be good enough to be able to refine. To do that, some systems have taken to using a mixed-initiative approach.

Mixed-initiative approaches let the designer and system take turns when generating assets which can lead to the refinement and generation phase to interweave, allowing for faster asset production [10]. *Tanagra* [11], a mixed-initiative system, allows for the user to take turns with the machine, choosing how a level will be generated. This allows for the user to put their own domain

knowledge into the system without having to encode it through the rule-based process described earlier. A downside is that mixed initiative tools are still domain-specific, such as a level generator can only be a level generator unless modified at the technical level. This leads to the designer once again must have a technical background to effectively transfer the system. In attempt to decrease this technical bottleneck, we look at other disciplines for insight. The discipline we look for guidance is architecture.

Games and architecture have an interesting relationship in terms of design. An example of architecture meshing well with games would be work done with the game *The Witness* [12]. Jonathan Blow brought on architects to help create a more cohesive and unique world which helped make the game a commercial success. By having this cross talk, the world within *The Witness* was able to be realized more so than had it been just game developers. This has also happened in the case of *Toon Town* in which game designer, Jesse Schell pulled information from books from the architecture domain to create *Toon Town’s* neighborhoods [13]. This same pattern can be seen in the *Sims* as well. By pulling knowledge from architecture books, the *Sims* encoded design patterns to help give more interesting gameplay, such as house layout and the effects on mood [14]. However, we believe these works and domain knowledge are still too specific, it can lead to a strong dependency between the system and the knowledge base. Instead, we look at Christopher Alexander’s *The Nature of Order* to use as the foundational stepping stone for a knowledge base.

3 Technical Description

This section will be focused on describing the different properties that are mentioned in Alexander’s *The Nature of Order*. To help ground these concepts, we will be using *Defense of the Ancients 2* (DotA 2), a Multiplayer Online Battle Arena (MOBA) developed by Valve [15]. Alexander points out fifteen properties of wholeness. We will only be discussing three of these properties because we found them to be good candidates in terms of translating them over to a knowledge base. The three candidate properties are strong centers, boundaries, and contrast.

3.1 Strong Centers

Strong centers are the heart of whatever system they are in. Christopher Alexander describes strong centers as:

“... the most important feature of a thing which is alive is that we find that the various wholes which exist at different levels appear not merely as centers or “wholes” or “blobs,” but actually as strong centers.” [16]

In DotA 2, strong centers come from the building within the map. The strongest centers of these buildings are the ancients. These ancients give rise to the main objectives of DotA 2. As teams are split into two camps, the Dire and the Radiant, players are given an ancient which they are allied to, but also an ancient which they must destroy.



Figure 1: Dire Ancient surrounded by support structures from Dota 2, an example of strong centers.

A team wins if they destroy the opposing ancient. In order to signal how important these ancients are, the game map shows it by making them some of the strongest centers in the game. The other strong centers come from how the function which they give players, such as shops, towers, or even boss monsters. However, only the ancients, towers, and characters are given priority in terms of game significance. Figure 1 shows the Dire ancient within the Dota 2 map. The ways the structures are placed give the Dire ancient even more sense of importance.



Figure 2: Left: Dota 2 minimap with markers denoting players and buildings. Right: Dota 2 minimap without markers.

The ancients give the most prominent example of how the level design and strong centers can draw the player into knowing what is important. By looking at the minimap of Dota 2 in Figure 2, there are other important locations to note, such as the red and green squares. Each of these squares represents some objective such as the ancient or other buildings that the players must either defend or destroy. The other symbols on the minimap are player characters which are not important for this example; however, do notice each player gravitates towards one of these squares. If someone were to look at this minimap, they may infer that the squares hold some importance to the game world.

3.2 Boundaries

If strong centers are the heart of the system, boundaries show where one system ends and another begins, and thus help bridge

each distinct system together to create cohesion. Christopher Alexander describes boundaries as:

“... The way that the field-like effect of a center is strengthened by the creation of a ring-like center, made of smaller centers that surround and intensify the former. The boundary also unites the center with the centers beyond it, thus strengthening it further.” [16]

These boundaries serve to help give some guidance on what is apart of a system and what is not. This allows observers to see where one thing ends and another begins. However, Christopher Alexander describes each of these systems to be interwoven to create something whole. To try and bring that to an example, Dota 2 has a game mechanic called neutrals. By being agnostic of a side, these neutrals can attack and be attacked by any character within the game. The neutrals are ranked in terms of difficulty to defeat. Harder neutrals give higher rewards. Among the hardest neutrals to defeat, a creature named Roshan has its own pit, separated from other parts through a boundary. In figure 3, we can see the pit which Roshan is placed as well as the barriers around the pit. These barriers act as a boundary for Roshan from the rest of the map. In fact, on the minimap in figure 2 the pit can be seen in the middle left, however, the contrast of colors can make it hard to see.



Figure 3: Roshan's pit, an example of boundaries within Dota 2.

This boundary gives a clear indication that there is something for the player to interact with. In this instance, that thing is the strongest neutral within the game.

3.3 Contrast

The last of the three concepts is contrast, or as Christopher Alexander describes as:

“The way that a center is strengthened by the sharpness of the distinction between its character and the character of surrounding centers.” [16]

By adding in contrast, systems can become even more distinct from one another. This property shows itself on the minimap in figure 2. By having a dull blue background, the red and green squares can stand out, even more, thus drawing the player's eye to more important parts on the map. While there are black borders around these squares, the distinct contrast between colors helps guide the

player's eye. Even with the location of Roshan, the boundaries that were previously discussed can be harder to see that the squares on the map.



Figure 4: Towers from both Radiant and Dire, an example of contrast.

Through contrast, systems such as towers and ancients can be discerned quickly by which color, design, and shape. Figure 4 shows this by comparing the two different tower schemes which the players see within the game world. The Dire tower has more of a dark take on what a tower should look like, which echoes the design of the Dire ancient shown in Figure 1. While the Radiant tower contrasts this by using a different color scheme and even a completely different design.

It should be noted, that while these three properties were briefly explained, each one could be seen working together. Such as with figure 4, even though the towers were used to describe contrast, they also incorporate boundaries and strong centers. Since they are so interwoven with each other, we believe this is good candidates for knowledge encoding. The same goes for the ancient in figure 1, the minimap in figure 2, and Roshan's pit in figure 3.

3.4 Wave Function Collapse

In order to encode these properties, we have chosen a candidate PCG algorithm, Wave Function Collapse (WFC). This algorithm has already seen commercial adoption via independent channels. Games such as *Bad North* [17] and *Caves Of Qud* [18] have used variants of WFC to generate levels during gameplay[19]. As for why WFC was chosen over other methods such as random generation or search based algorithms, WFC has some properties that are appealing such as its probabilistic choice and constraint reasoning. These two properties give WFC the ability to choose and then take out any incompatible solutions for asset generation. The issue with this is there can be a chance in which WFC is unable to find a satisfiable solution. Since WFC can be a non-deterministic

algorithm, as in the output isn't always the same, by running WFC again there is a chance of finding a satisfiable solution. The usual cause for an unsatisfiable solution comes from an overly constrained set of rules, as in the rules are too restrictive for the algorithm to find something that satisfies all the rules. This constraint solving portion is what we are more interested in, because it will allow us to encode these properties from Alexander's *The Nature of Order* into WFC.

As for how to encode these properties, we are still experimenting with possible solutions. Our first approach is to encode these directly as rules, however, this can be quite clunky and may not lend itself to ease of use. It might lead to where developers are attempting to figure out which of the rules might be a more domain specific rule vs a more generalized rule like a contrasting rule. In order to alleviate this, we are trying to use higher level rules for higher level properties such as strong centers, boundaries, and contrast. We believe that since these properties are higher level concepts should be associated with their own higher-level rules. These rules will still work in conjunction with other more domain specific rules, as a key placement rule and a contrasting rule can work together. The goal here is to try and decouple them so the more domain-specific rules can be swapped out while higher level rules remain the same.

4 Limitations

A fundamental limitation of this work is proving that these properties will hold for all domain spaces. Another limitation of this work is if there is an implicit notion of these properties in everything then why encode these more general properties into a PCG system. We argue that these properties are a foundational step in which we can walk towards more domain-specific encoding but do so in a hierarchical approach that gives way to more reusability to PCG systems. Another limitation is seeing how much technical knowledge is still required to properly use this system.

5 Conclusion/Future Works

In this paper, we describe three candidate properties from Alexander's work *The Nature of Order* and give a partial solution for encoding them into PCG systems. We give three properties and a candidate algorithm to build a guideline for how we think domain-specific knowledge should be encoded. By starting at the general and funneling down towards the more specific. However, we have not fully experimented with this framework to give strong statements such as it will solve the upfront time cost that other domain-specific PCG algorithms have and further testing needs to be done. We do propose a framework that motivates a different view of how to encode knowledge within PCG algorithms. As for future works, we aim to develop a mixed-initiative system that can both help users design more unique assets and help write rules for the domain specific tasks.

REFERENCES

- [1] G. Smith, "Understanding procedural content generation: a design-centric analysis of the role of PCG in games," 2014, pp. 917–926.
- [2] G. Smith, "The Future of Procedural Content Generation in Games," p. 5.
- [3] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, "Search-Based Procedural Content Generation: A Taxonomy and Survey," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 172–186, Sep. 2011.
- [4] A. M. Smith and M. Mateas, "Answer Set Programming for Procedural Content Generation: A Design Space Approach," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 187–200, Sep. 2011.
- [5] D. B. Lenat, R. V. Guha, K. Pittman, D. Pratt, and M. Shepherd, "Cyc: toward Programs with Common Sense. Cyc: toward Programs with Common Sense Motivation: The," 2006.
- [6] D. Ramachandran, "First-Orderized ResearchCyc: Expressivity and Efficiency in a Common-Sense Ontology," p. 8.
- [7] W. Knight, "An AI that spent 30 years learning some common sense is ready for work," *MIT Technology Review*. [Online]. Available: <https://www.technologyreview.com/s/600984/an-ai-with-30-years-worth-of-knowledge-finally-goes-to-work/>. [Accessed: 19-Apr-2019].
- [8] N. Shaker, J. Togelius, and M. J. Nelson, "Fractals, noise and agents with applications to landscapes," in *Procedural Content Generation in Games*, Cham: Springer International Publishing, 2016, pp. 57–72.
- [9] C. Michael, "Possibility Space." [Online]. Available: <http://www.possibilityspace.org/tutorial-generative-possibility-space/>. [Accessed: 19-Apr-2019].
- [10] E. Butler, A. M. Smith, Y.-E. Liu, and Z. Popovic, "A mixed-initiative tool for designing level progressions in games," in *Proceedings of the 26th annual ACM symposium on User interface software and technology - UIST '13*, St. Andrews, Scotland, United Kingdom, 2013, pp. 377–386.
- [11] G. Smith, J. Whitehead, and M. Mateas, "Tanagra: Reactive Planning and Constraint Solving for Mixed-Initiative Level Design," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 201–215, Sep. 2011.
- [12] B. Jonathan, "Architecture in The Witness – The Witness." [Online]. Available: <http://the-witness.net/news/2011/12/architecture-in-the-witness/>. [Accessed: 19-Apr-2019].
- [13] S. Jesse, "The Nature of Order in Game Narrative." [Online]. Available: <https://www.gdcvault.com/play/1025006/The-Nature-of-Order-in->. [Accessed: 19-Apr-2019].
- [14] D. Chan, "THE PHILOSOPHY OF THE SIMS," p. 12.
- [15] Valve, *Defense of the Ancients 2*. 2013.
- [16] A. Christopher, *The Nature of Order: An Essay on the Art of Building and The Nature of the Universe, Book One, the Phenomenon of Life*. Berkeley, California: Center For Environmental Structure, 2002.
- [17] Plaisible Concept, *Bad North*. 2018.
- [18] Freehold Games, *Caves of Qud*. 2019.
- [19] I. Karth and A. M. Smith, "WaveFunctionCollapse is constraint solving in the wild," in *Proceedings of the International Conference on the Foundations of Digital Games - FDG '17*, Hyannis, Massachusetts, 2017, pp. 1–10.